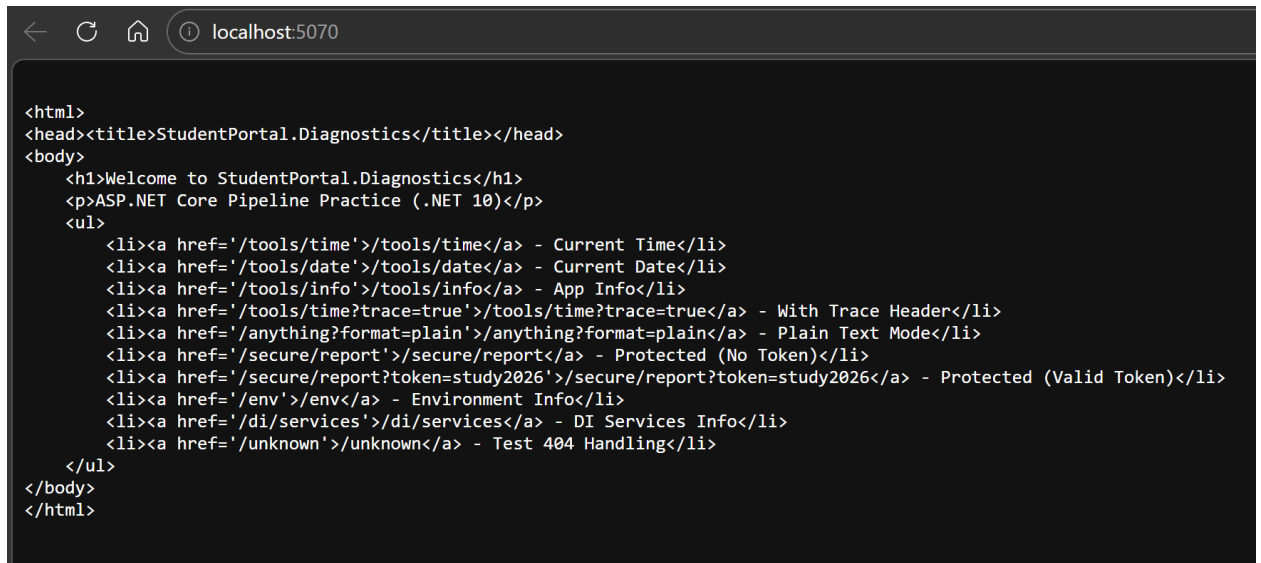


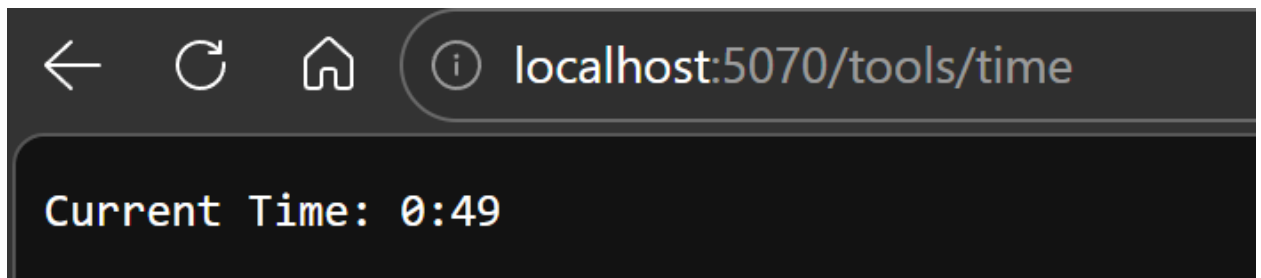
1. Скриншоты успешных запросов к обязательным маршрутам

Маршрут: /



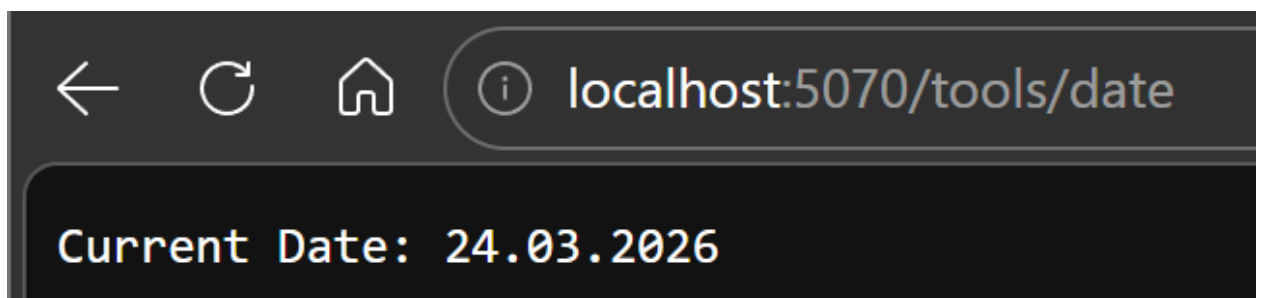
```
<html>
<head><title>StudentPortal.Diagnostics</title></head>
<body>
  <h1>Welcome to StudentPortal.Diagnostics</h1>
  <p>ASP.NET Core Pipeline Practice (.NET 10)</p>
  <ul>
    <li><a href='/tools/time'/>tools/time</a> - Current Time</li>
    <li><a href='/tools/date'/>tools/date</a> - Current Date</li>
    <li><a href='/tools/info'/>tools/info</a> - App Info</li>
    <li><a href='/tools/time?trace=true'/>tools/time?trace=true</a> - With Trace Header</li>
    <li><a href='/anything?format=plain'/>anything?format=plain</a> - Plain Text Mode</li>
    <li><a href='/secure/report'/>secure/report</a> - Protected (No Token)</li>
    <li><a href='/secure/report?token=study2026'/>secure/report?token=study2026</a> - Protected (Valid Token)</li>
    <li><a href='/env'/>env</a> - Environment Info</li>
    <li><a href='/di/services'/>di/services</a> - DI Services Info</li>
    <li><a href='/unknown'/>unknown</a> - Test 404 Handling</li>
  </ul>
</body>
</html>
```

Маршрут: /tools/time



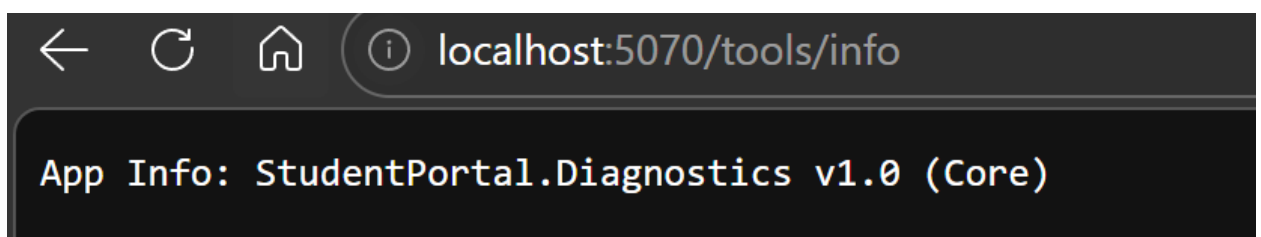
```
Current Time: 0:49
```

Маршрут: /tools/date



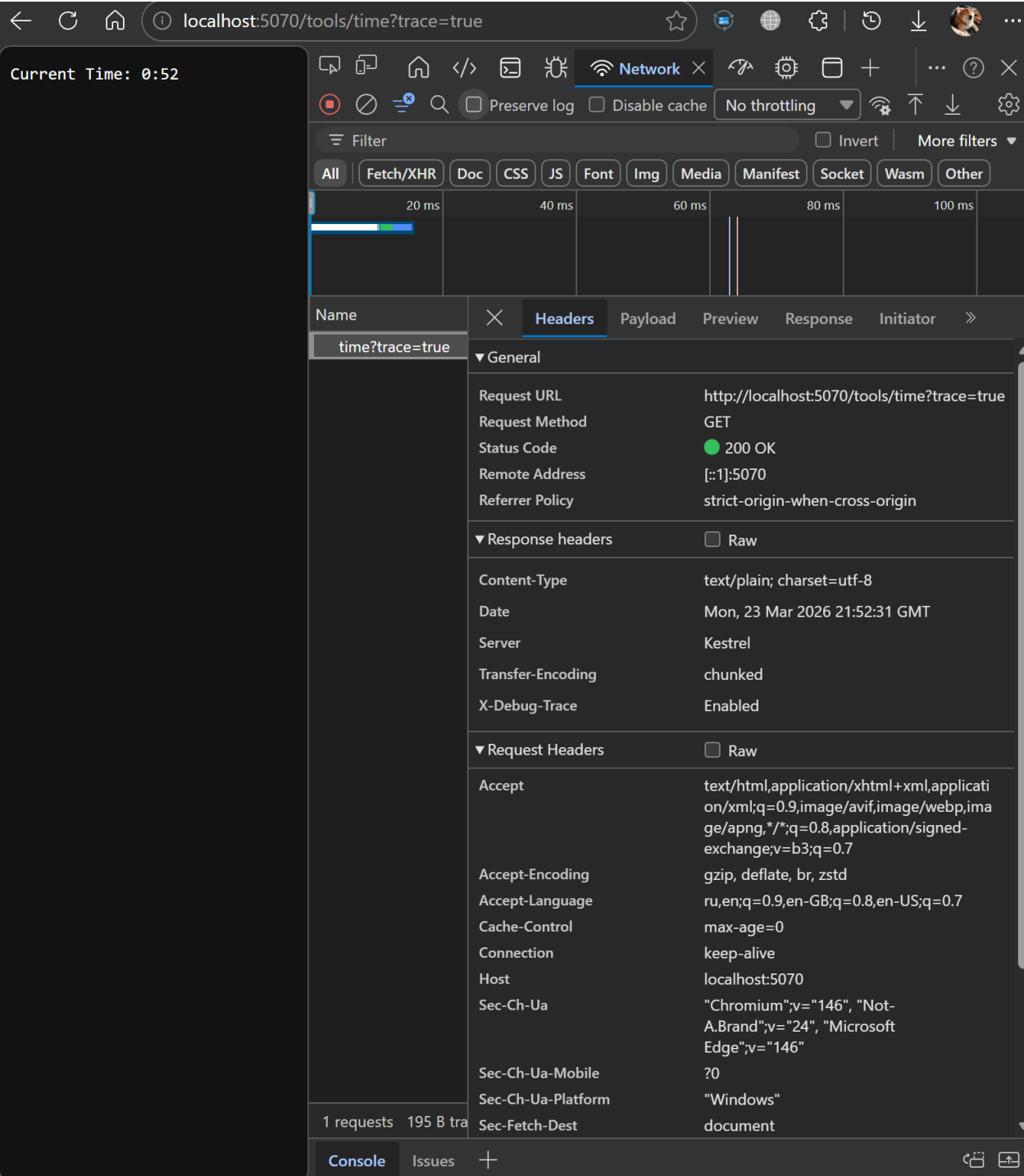
```
Current Date: 24.03.2026
```

Маршрут: /tools/info

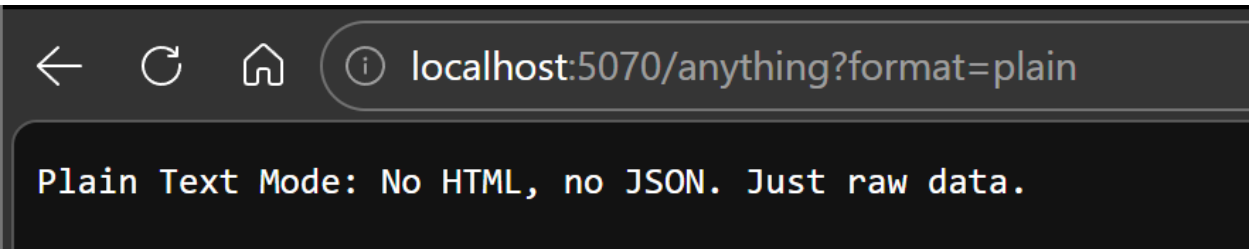


```
App Info: StudentPortal.Diagnostics v1.0 (Core)
```

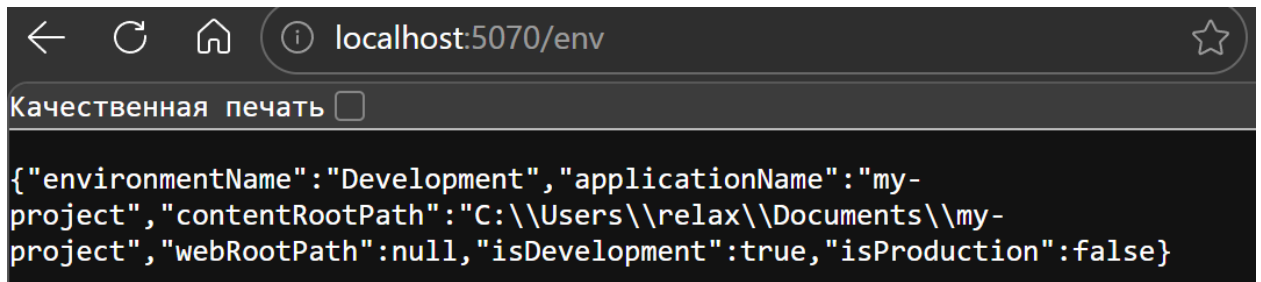
Маршрут: /tools/time?trace=true



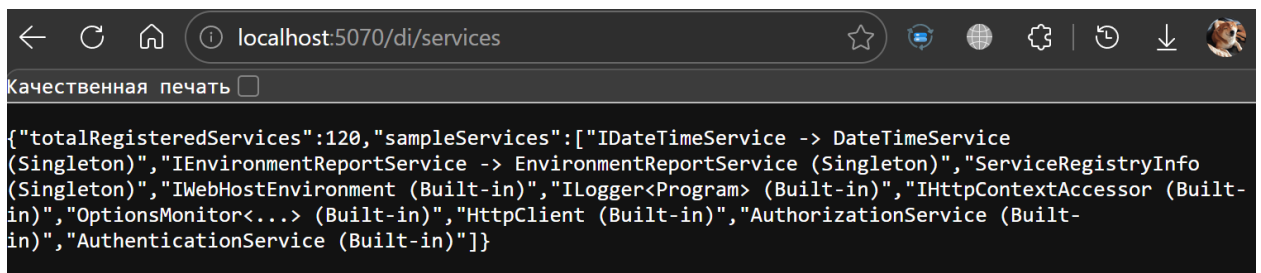
Маршрут: /anything?format=plain



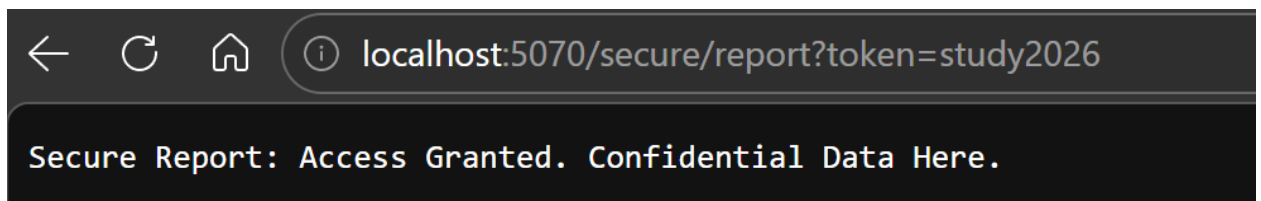
Маршрут: /env



Маршрут: /di/services

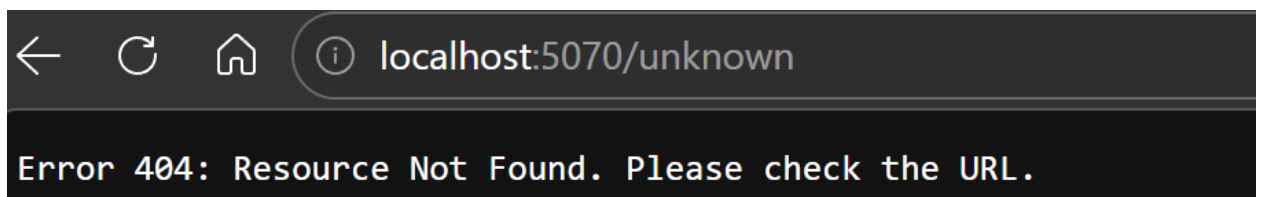


Маршрут: /secure/report?token=study2026

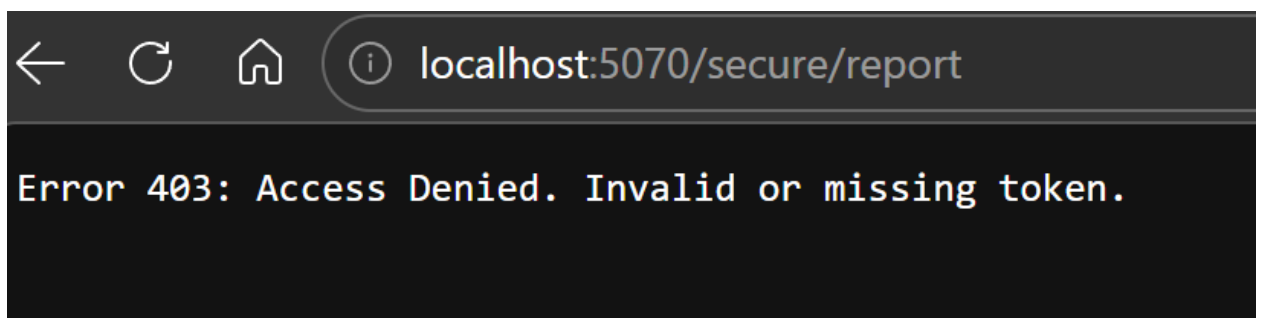


2. Скриншот /unknown и /secure/report без токена

Маршрут: /unknown



Маршрут: /secure/report



3. Фрагмент Program.cs с итоговым порядком middleware

```
var app = builder.Build();
```

```
app.UseMiddleware<ErrorHandlingMiddleware>();
```

```
app.Use(async (context, next) =>
```

```
{
```

```
    var startTime = DateTime.Now;
```

```
    await next();
```

```
    var duration = DateTime.Now - startTime;
```

```
    if (!context.Response.HasStarted)
```

```
    {
```

```
        context.Response.Headers["X-Processing-Time"]
```

```
=
```

```
$"{duration.TotalMilliseconds} ms";
```

```
    }
```

```
});
```

```
app.UseWhen(
```

```
    context
```

```
=>
```

```
    context.Request.Query.ContainsKey("trace")
```

```
&&
```

```
    context.Request.Query["trace"] == "true",
```

```
    appBranch =>
```

```
{
```

```

appBranch.Use(async (context, next) =>
{
    context.Response.Headers["X-Debug-Trace"] = "Enabled";

    await next();

});

});

app.MapWhen(
    context => context.Request.Query.ContainsKey("format") &&
    context.Request.Query["format"] == "plain",
    appBranch =>
    {
        appBranch.Run(async context =>
        {
            context.Response.ContentType = "text/plain";

            await context.Response.WriteAsync("Plain Text Mode: No HTML, no
JSON. Just raw data.");

        });
    }
);

app.MapGet("/tools/time", (IDateTimeService timeService) =>

```

```

    $"Current Time: {timeService.GetTime()}");

app.MapGet("/tools/date", (IDateTimeService timeService) =>

    $"Current Date: {timeService.GetDate()}");

app.MapGet("/tools/info", (IEnvironmentReportService envService) =>

    $"App Info: {envService.GetAppInfo()}");


app.Map("/secure", secureApp =>
{
    secureApp.UseToken("study2026");

    secureApp.Map("/report", reportApp =>
    {
        reportApp.Run(async context =>
        {
            context.Response.ContentType = "text/plain";

            await context.Response.WriteAsync("Secure Report: Access Granted.
Confidential Data Here.");

        });
    });

    secureApp.Map("/admin/report", adminApp =>
    {
        adminApp.Run(async context =>
        {
            context.Response.ContentType = "text/plain";

```

```
        await context.Response.WriteAsync("Admin Secure Report: Top Secret  
Data.");  
  
    });  
  
});  
  
});
```

```
app.MapGet("/env", (IWebHostEnvironment env) =>  
{  
    return new  
  
    {  
        EnvironmentName = env.EnvironmentName,  
        ApplicationName = env.ApplicationName,  
        ContentRootPath = env.ContentRootPath,  
        WebRootPath = env.WebRootPath,  
        IsDevelopment = env.IsDevelopment(),  
        IsProduction = env.IsProduction()  
    };  
});
```

```
app.MapGet("/di/services", (ServiceRegistryInfo info) =>  
{  
    var result = new  
  
    {
```

```
TotalRegisteredServices = info.TotalCount,  
  
SampleServices = info.ServiceDescriptions.Take(10)  
  
};  
  
return result;  
  
});  
  
app.MapGet("/", () => @"<html>...");  
  
app.Run();
```

4. Краткое объяснение разницы между UseWhen и MapWhen

Главное различие между UseWhen и MapWhen заключается в механизме возврата управления в основной конвейер обработки запросов.

UseWhen создает временную ветку middleware. Если условие выполнено (в нашем проекте — наличие параметра `trace=true`), выполняется дополнительный код (например, добавление заголовка отладки), после чего обязательно вызывается `next()`. Это возвращает запрос в основной pipeline, позволяя ему достигнуть целевого эндпоинта. Такой подход используется для расширения функциональности (логирование, диагностика) без прерывания основной логики приложения.

MapWhen создает независимую, изолированную ветку конвейера. Если условие истинно (в проекте — параметр `format=plain`), запрос обрабатывается исключительно внутри этой ветки (обычно через терминальный метод `Run()`), и основной конвейер ниже точки ветвления не выполняется. Это применяется для полной замены логики обработки для определённых типов запросов.

5. Объяснение, почему ErrorHandlerMiddleware должен стоять раньше остальных

Основная причина заключается в архитектуре конвейера обработки запросов, где middleware работают по принципу вложенной обертки. Чтобы корректно перехватывать ошибки (404 и 403), возникающие в downstream-компонентах, обработчик ошибок должен окружать их со всех сторон.

Механизм работы строится вокруг вызова `next()`. Middleware сначала фиксирует начало, затем вызывает `await next()`, передавая управление всем последующим компонентам (маршрутизации, аутентификации, эндпоинтам). Только после того, как все они отработают и вернут управление обратно, `ErrorHandlingMiddleware` проверяет статус ответа. Если статус равен 404 или 403, он модифицирует тело ответа, добавляя понятное сообщение пользователю.

Если разместить этот компонент в конце конвейера, он потеряет возможность перехвата. К моменту его запуска ответ уже будет сформирован предыдущими компонентами и отправлен клиенту, либо статус код будет установлен без возможности изменения. В таком случае пользователь увидит стандартную системную ошибку вместо дружественного сообщения.

6. Перечень собственных сервисов и место их регистрации

6.1. Зарегистрированные сервисы

В проекте зарегистрировано три собственных сервиса:

`IDateTimeService` - интерфейс для получения текущей даты и времени. Реализация `DateTimeService` предоставляет методы `GetDate()` и `GetTime()`. Зарегистрирован как `Singleton`, поскольку время системы - это глобальное состояние, не требующее создания нового экземпляра на каждый запрос.

`IEnvironmentReportService` - интерфейс для получения диагностической информации о приложении. Реализация `EnvironmentReportService` возвращает название и версию приложения. Зарегистрирован как `Singleton`, так как информация о приложении не меняется в течение его работы.

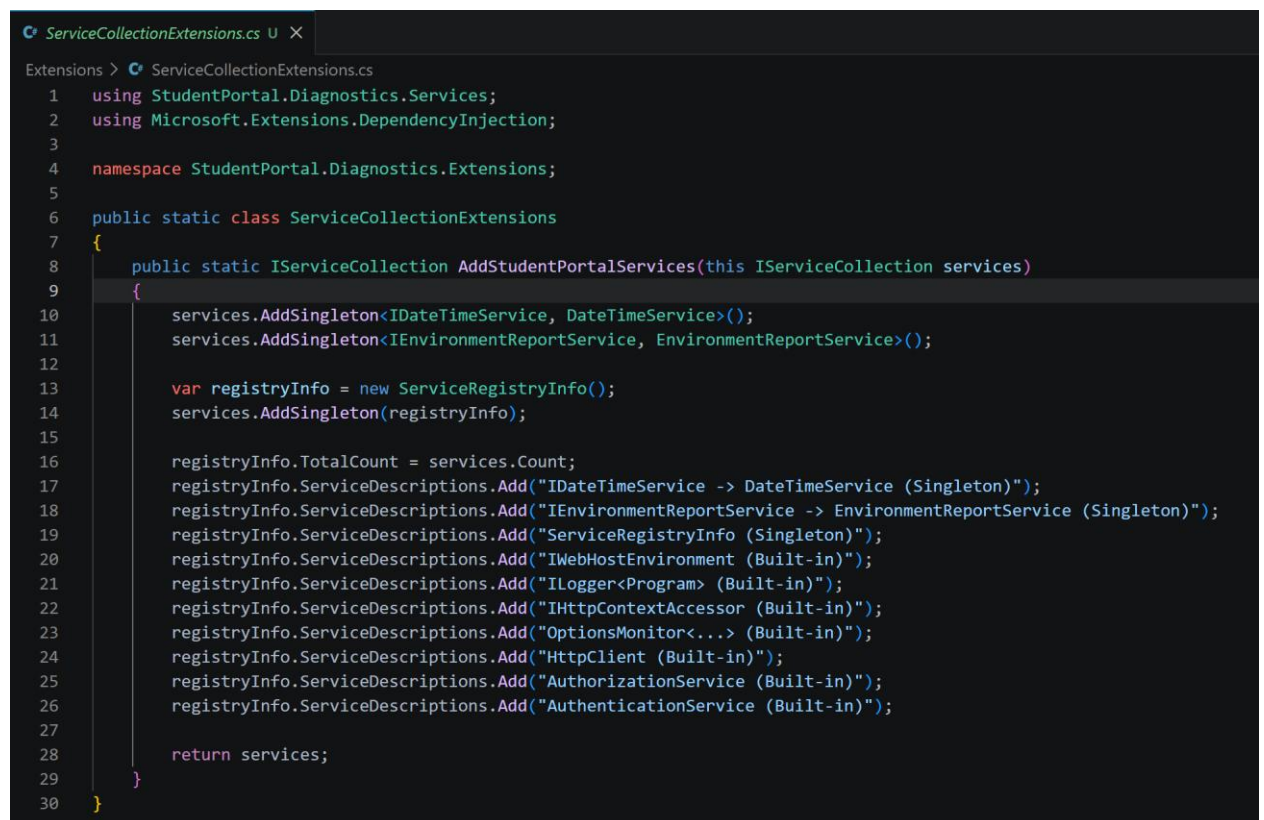
ServiceRegistryInfo - класс для хранения сведений о зарегистрированных сервисах. Используется для вывода информации на эндпоинте /di/services. Зарегистрирован как Singleton, чтобы накапливать и хранить данные о коллекции сервисов в течение всего времени работы приложения.

6.2. Место регистрации

Регистрация сервисов вынесена в отдельный класс расширений для IServiceCollection.

Файл: Extensions/ServiceCollectionExtensions.cs

Метод расширения: AddStudentPortalServices()



```
ServiceCollectionExtensions.cs U X
Extensions > ServiceCollectionExtensions.cs
1 using StudentPortal.Diagnostics.Services;
2 using Microsoft.Extensions.DependencyInjection;
3
4 namespace StudentPortal.Diagnostics.Extensions;
5
6 public static class ServiceCollectionExtensions
7 {
8     public static IServiceCollection AddStudentPortalServices(this IServiceCollection services)
9     {
10         services.AddSingleton<IDateTimeService, DateTimeService>();
11         services.AddSingleton<IEnvironmentReportService, EnvironmentReportService>();
12
13         var registryInfo = new ServiceRegistryInfo();
14         services.AddSingleton(registryInfo);
15
16         registryInfo.TotalCount = services.Count;
17         registryInfo.ServiceDescriptions.Add("IDateTimeService -> DateTimeService (Singleton)");
18         registryInfo.ServiceDescriptions.Add("IEnvironmentReportService -> EnvironmentReportService (Singleton)");
19         registryInfo.ServiceDescriptions.Add("ServiceRegistryInfo (Singleton)");
20         registryInfo.ServiceDescriptions.Add("IWebHostEnvironment (Built-in)");
21         registryInfo.ServiceDescriptions.Add("ILogger<Program> (Built-in)");
22         registryInfo.ServiceDescriptions.Add("IHttpContextAccessor (Built-in)");
23         registryInfo.ServiceDescriptions.Add("OptionsMonitor<...> (Built-in)");
24         registryInfo.ServiceDescriptions.Add("HttpClient (Built-in)");
25         registryInfo.ServiceDescriptions.Add("AuthorizationService (Built-in)");
26         registryInfo.ServiceDescriptions.Add("AuthenticationService (Built-in)");
27
28         return services;
29     }
30 }
```

6.3. Вызов регистрации в Program.cs

Регистрация выполняется до вызова builder.Build(), что является обязательным требованием для корректной работы DI-контейнера.

Файл: Program.cs

```
6    var builder = WebApplication.CreateBuilder(args);  
7  
8    builder.Services.AddStudentPortalServices();  
9    builder.Services.AddSingleton(builder.Environment);  
10  
11   var app = builder.Build();
```

7. Вывод по работе: что было самым важным в понимании request pipeline.

Самым важным в понимании конвейера обработки запросов стало осознание того, что middleware работают по принципу вложенной обёртки, где порядок регистрации определяет последовательность выполнения.

Первый ключевой инсайт - порядок имеет критическое значение. Middleware регистрируются в Program.cs сверху вниз, но выполняются в двух фазах: до next() при движении запроса внутрь и после next() при возврате ответа. Это означает, что компонент, зарегистрированный первым, получает запрос первым, а ответ - последним. Именно поэтому ErrorHandlerMiddleware должен стоять в начале — только так он сможет перехватить статусы 404 и 403, установленные downstream-компонентами.

Второй важный урок - различие между терминальным и нетерминальным middleware. Терминальный обработчик (Run) завершает конвейер и не вызывает next(), поэтому всё, что зарегистрировано после него, никогда не выполнится. Нетерминальный обработчик (Use) обязан вызвать next(), иначе запрос «повиснет» и клиент не получит ответ. Понимание этой разницы необходимо для правильного ветвления pipeline.

Третий инсайт - механизм ветвления через UseWhen и MapWhen работает принципиально по-разному. UseWhen расширяет основной конвейер с обязательным возвратом, тогда как MapWhen создаёт изолированную ветку, полностью перехватывающую запрос. Ошибка в выборе метода приводит к тому, что либо запрос не доходит до целевого эндпоинта, либо выполняется лишняя работа.

Четвёртый урок - зависимость между middleware и DI-сервисами. Сервисы регистрируются до `builder.Build()`, а middleware настраиваются после. Это разделение гарантирует, что все зависимости доступны на момент запуска приложения, но требует дисциплины в организации кода.

В итоге, Request Pipeline - это крутая штука, а не просто набор функций - строго упорядоченная цепочка обработчиков, где каждый компонент влияет на последующие. Неправильный порядок middleware может сделать приложение неработоспособным или небезопасным, даже если весь код синтаксически верен.